

MuonCollider BIB analysis: user manual

How to run the package, where everything lives, and what comes out. Luciano Ristori, October 2026.

Master copy: `~/code/MuonCollider/docs/user_manual.pdf` (git; source docs/src/user_manual/user_manual.tex).

The copy `_START_HERE_user_manual.pdf` in the Analysis folder is refreshed from it automatically by `./run_all` and `./run_scan`: do not edit the copy; change the source and rebuild (see `README.txt` next to it).

The 30-second version. Everything is run from *one* folder, in a Terminal:

```
cd ~/Dropbox/Documents/MuonColliderSimulation/Analysis
```

In that folder you edit plain-text settings files (names starting with `__`), then type one command:

- `./run_all` — applies smearing and cuts to the BIB and signal files and makes the highlights plots and PDF, with the BIB rejection, signal efficiencies *and the expected number of fake tracks* for your cuts and resolutions (Sections 3 and 4).
- `./run_scan` — scans the timing or pointing-angle resolution, re-deriving the cuts at each point (Section 5).

Every run is archived, with a copy of the settings it used, in `runs/`. The newest results are always the files whose names start with `_` in the Analysis folder. Nothing you do there can damage the code.

1 Where everything lives

There are two places: the **code**, under git, and the **simulation folder** in Dropbox, which holds the data, the settings and all results.

1.1 The simulation folder (Dropbox) — where you work

```
~/Dropbox/Documents/MuonColliderSimulation/
|-- Data/                input ROOT n-tuples (BIB plus/minus/ipp, combined BIB, signal)
|-- Geometry/            detector geometry XML files (MuSIC, Vertex, IT, OT)
'-- Analysis/            <-- THE WORKING FOLDER: cd here, edit, run
    |-- __input_files_config.txt  which files in Data/ and Geometry/ to use
    |-- __cuts_config.txt        cuts per subsystem, track definition
    |-- __smearing_config.txt    detector resolutions (position, time, angle)
    |-- __scan_config.txt        what ./run_scan scans
    |-- run_all, run_scan        the two commands (tiny launchers)
    |-- _START_HERE_user_manual.pdf  this manual (auto-refreshed copy)
    |-- _highlights_<date>_<time>.pdf  PDF of the latest ./run_all
    |-- _highlights/            its key plots (PNG) and the same PDF
    |-- _scan_time_<date>_<time>.pdf  PDF of the latest time scan
    |-- _scan_angle_<date>_<time>.pdf  PDF of the latest angle scan
    |-- latest_run.txt          which run the step* folders show, + its summary
    |-- step1... step4...       all plots/tables of the latest successful ./run_all
    |-- runs/                  ARCHIVE: one folder per run or scan (see Sec. 6)
    '-- claude_scratch/        scratch files from work with Claude; ignore
```

1.2 The code (git) — normally you don't touch it

```
~/code/MuonCollider/          git, github.com/LucianoRistori/MuonCollider
|-- README.md                 detailed reference: n-tuple format, every step, definitions
|-- docs/user_manual.pdf      this manual (source in docs/src/user_manual/)
|-- docs/efficiency_pt_infinity.pdf  note: how efficiencies for pT -> inf are obtained
|-- run_all_steps.sh          what ./run_all does (checks, steps 1-4, archiving)
|-- run_scan.sh               what ./run_scan does
|-- bib_common.py             shared library: reading, smearing, cuts, efficiencies
|-- make_basic_plots.py, incidence_angle_plots.py, time_of_flight_plots.py,
|   signal_overlay_*.py, apply_cuts.py, track_efficiency.py, nl_cut_plots.py
|                             the analysis steps 1-4
|-- fake_rate.py              expected number of fake tracks (Sec. 4)
|-- resolution_scan.py        the resolution scans (Sec. 5)
|-- make_highlights_pdf.py    builds the highlights PDF of a run
|-- check_configs.py, cuts_table.py, input_files.py  read and check the settings
|-- templates/                pristine copies of the four settings files
|-- fake_rate_framework/      analytic fake-track framework of the paper (Sec. 13-14)
'-- resolution_scans/         original scripts and results of paper Sec. 15-16
```

The launchers `run_all` and `run_scan` in the Analysis folder just find the code in `~/code/MuonCollider` and run it on the folder they are in. If you move the code, run with `MUONCOLLIDER_CODE=/new/path` `./run_all`.

2 The four settings files

All four are commented plain text: open them in any editor and read the comments, which explain every line. A summary:

<code>__input_files_config.txt</code>	[data]: the BIB files bib_plus, bib_minus, bib_ipp (optional), the combined BIB file bib_combined, and the signal muon-gun sample; [geometry]: the four XML files. <i>Names only</i> : files are looked up in Data/ and Geometry/. The combined file is (re)built automatically when it is missing or does not match plus + minus + ipp.
<code>__cuts_config.txt</code>	[cuts]: a table, one row per subsystem (VXD/IT/OT, barrel and endcap), with the three cuts <i>time</i> ($ t_{\text{corrected}} $, ns), <i>z0</i> ($ z_0 $, mm) and <i>pT</i> (minimum p_T , GeV/c), or <i>off</i> ; then three <i>zoom</i> columns that only set the range of the zoomed N-1 plots. [track]: a muon is “found” if at least min_hits_found of its hits pass (exclude_vertex_hits: VXD hits not counted). [signal]: muon_hits_only.
<code>__smearing_config.txt</code>	Gaussian resolutions applied to the (exact) simulated hits: [position] sigma_u_mm, sigma_v_mm; [time] sigma_t_ns; [angle_u], [angle_v] sigma_deg (pointing angle). Each section has enabled = true/false. [general] seed fixes the random numbers.
<code>__scan_config.txt</code>	Only for ./run_scan: parameter (time or angle), containment (% of signal hits each re-derived cut keeps) and grid (the values to scan).

Both commands check the files before running anything (typos, unknown settings, non-numbers, missing input files) and stop with a message saying what to fix. If a settings file is missing, a fresh copy is made from templates/ and you are asked to check it. Editing a file while a run is going has no effect on that run: the files are copied at the start.

3 Case 1 — highlights plots for some data files

1. New data? Copy the ROOT files into Data/ and put their names in `__input_files_config.txt`. (Same data as last time: skip this.)
2. Set the cuts in `__cuts_config.txt` and the resolutions in `__smearing_config.txt`.
3. Run:

```
cd ~/Dropbox/Documents/MuonColliderSimulation/Analysis
./run_all          # step 4 only: everything the PDF needs  (~1 1/4 min)
./run_all --full   # steps 1-4: all plots of the analysis  (~6 min)
```

4. Look at `_highlights_<date>_<time>.pdf` in the Analysis folder.

What the steps are. Step 1: hit maps and densities of the BIB (plus, minus, combined). Step 2: incidence angles and time-of-flight of BIB and signal. Step 3: BIB and signal overlaid (z_0 , p_T , corrected time). Step 4: the cuts — BIB rejection and signal hit efficiency per subsystem, track-finding efficiency vs. p_T , N-1 plots, expected fake tracks.

What comes out. The PDF has a title page with the settings and the main results (BIB rejection factor

and signal hit efficiency for $p_T \rightarrow \infty$ per subsystem, track-finding efficiency, expected fake tracks), the cuts per subsystem, and the key plots with captions quoting that run's numbers. The same results are in `summary.txt` of the run (also at the bottom of `latest_run.txt`). The `step*` folders and `_highlights/` show the latest *successful* run; a failed run never replaces them. To compare all past runs: `cat runs/*/summary.txt`.

4 Case 2 — fake-track rate for given cuts and resolutions

This is the same command as Case 1: set `__cuts_config.txt` and `__smearing_config.txt`, then `./run_all`. The expected number of fake tracks is reported

- on the title page of the highlights PDF (“Expected fake tracks”),
- in `summary.txt` and `latest_run.txt` (last line),
- in `step4_cuts/fake_rate.csv`, with the BIB hits after the cuts in each of the six layers used.

What the number is. $E[\text{\#fakes}]$ is the expected number of fake tracks in the 6-layer IT + OT barrel “tower” of the fake-rate framework (paper, Sections 14–16):

$$E[\text{\#fakes}] = P_{\text{true}} \prod_{i=1}^6 n_i, \quad n_i = \frac{\text{BIB hits after the cuts in layer } i}{\text{area of layer } i} \times W_i^2,$$

where W_i^2 are the plane areas of the projective tower (1 m² outermost plane) and P_{true} is the fake probability calibrated in `fake_rate_framework/real_tower_calibration.py`, for three track models: exact helix (the one quoted), conservative quadratic, and straight line ($B = 0$). Values well below 1 mean fakes are negligible.

Headroom. Since $E[\text{\#fakes}]$ is a product of six densities, scaling the BIB density of all six layers by a common factor μ scales it by μ^6 ; one fake is reached at $\mu = E[\text{\#fakes}]^{-1/6}$. This factor is quoted on the title page (“1 fake at $N \times$ the BIB”), in the last line of `summary.txt`, in the `mu_one_fake_*` columns of `fake_rate.csv`, and plotted ($E[\text{\#fakes}]$ vs. μ , three models) in `fake_rate_vs_density_multiplier.png`, a page of the highlights. It tells how much more background (or looser cuts) the tracker can take before fakes matter.

Two things to remember. (1) It uses only the IT and OT barrel layers; the calibration assumes the current geometry (a warning is printed if the layer areas change). (2) There are two track-finding efficiencies. `./run_all` reports the *whole-tracker* one (all muons; found if at least `min_hits_found` of their hits pass, VXD hits not counted). `./run_scan` reports it too, and next to it the *barrel-tower* one, defined on the same six layers as $E[\text{\#fakes}]$ (muons with no IT/OT endcap hit; found if at least `min_hits_found` of their 6 barrel hits pass). They are different quantities: compare like with like.

5 Case 3 — resolution scans

1. Edit `__scan_config.txt`:

```
parameter = time           # or: angle
containment = 98           # % of signal hits kept by each re-derived cut
grid       = log 0.010 1.0 10 # time: 10 ps - 1 ns, 10 points
```

```
# angle:  grid = linear 0 5 11    (0 - 5 deg in 0.5 deg steps)
# any:    grid = list 0.02 0.05 0.1
```

2. Check `__smearing_config.txt` (all the resolutions that are *not* scanned) and the p_T cut in `__cuts_config.txt`.
3. Run `./run_scan` (about 15–25 s) and open `_scan_time_<date>_<time>.pdf` (or `_scan_angle_...`).

What a scan does at each value. The scanned resolution is applied to the hits of *all* subsystems (for angle, equally to u and v); every other resolution is taken from `__smearing_config.txt`; the p_T cut from `__cuts_config.txt` is kept fixed. The z_0 and time cuts of every subsystem are then *re-derived* so that each keeps containment % of the signal hits *in its own N-1 distribution*, exactly as in the N-1 plots of `./run_all`: z_0 on the hits passing p_T and time, time on those passing p_T and z_0 . The two are found together by iteration (2–4 rounds). The z_0 and time values written in `__cuts_config.txt` are *not* used. (Before October 2026 z_0 was set first, on the hits passing p_T only; late hits of curling low- p_T muons in the outer OT barrel layer then made the OT barrel z_0 cut about four times too loose.) The scan reports $E[\text{\#fakes}]$ and the track-finding efficiency for $p_T \rightarrow \infty$, both on the barrel tower, and also the whole-tracker efficiency exactly as `./run_all` defines it (Section 4).

What comes out (in `runs/<date>_<time>_scan_<time|angle>/`): a PDF with a settings page, a results table, and three plots ($E[\text{\#fakes}]$, the two efficiencies, and the derived cuts of all six subsystems vs. the resolution); `scan_results.csv` with every number. Scans made before the N-1 definition (and the paper’s Sections 15–16) have OT barrel z_0 cuts about four times looser, hence a higher $E[\text{\#fakes}]$.

Tip: `grid = list <value>` gives a single point: the efficiency and $E[\text{\#fakes}]$ for one resolution with containment-derived cuts, to compare with the fixed cuts of Case 2.

6 The archive: runs/

Every run gets its own folder, named by the date and time it started:

2026-10-07_161638	a full <code>./run_all --full run</code>
2026-10-07_161638_pdf	a default <code>./run_all</code> (step 4 only)
<code>.._scan_time, .._scan_angle</code>	a <code>./run_scan</code>
<code>.._FAILED, .._INTERRUPTED</code>	did not complete — see its <code>run_log.txt</code>

When a run finishes (or fails), all the files in its folder are made **read-only**, so the archive can’t be changed by accident; the folder itself can still be deleted. The copies in the `step*` folders and `_highlights/` stay editable.

Each holds the settings files exactly as used, `run_log.txt` (everything printed), `code_version.txt` (git commit of the code, input files with sizes and dates), and the results (`summary.txt`, `step` folders, `_highlights/`; or the scan’s CSV, plots and PDF). To repeat an old run, copy its settings files back into the Analysis folder and run again. The copies are read-only too: `chmod u+w __*_config.txt` makes them editable (each run also does this for you). To edit a file inside a run on purpose: `chmod u+w runs/<run>/<file>`. To rebuild the PDF of an old run: `python3 ~/code/MuonCollider/make_highlights_pdf.py runs/<run>`. Old run folders can be deleted freely; Dropbox keeps a version history.

7 If something goes wrong

- **“ERROR: ... fix the settings file(s)”** — nothing was run; the message lists each problem and the file it is in.
- **Python is missing packages** — the message gives the command, e.g. `python3 -m pip install uproot awkward`. Needed: `numpy`, `uproot`, `awkward`, `matplotlib`.
- **A run ends in `_FAILED`** — open its `run_log.txt`; the step that failed is at the end. Your `step*` folders and `_highlights/` still show the last good run.
- **`./run_all: Permission denied`** — the launcher lost its executable flag (e.g. after a copy):
`chmod +x run_all run_scan`.
- **Help on the commands** — `./run_all --help`, `./run_scan --help`.
- **A full run under a time limit** can be split: `./run_all --full --steps 1,2`, then `./run_all --run <run> --steps 3,4`.

8 Where to read more

- `README.md` in the code folder: the n-tuple format, each analysis step and its plots, how every quantity is defined and quoted.
- `docs/efficiency_pt_infinity.pdf`: the efficiencies for $p_T \rightarrow \infty$ (the fit of $\varepsilon_\infty + c/p_T^2$).
- The paper draft and the project notes (milestones, decisions) are in the Claude project “Muon Collider Tracking”.
- `fake_rate_framework/` and `resolution_scans/`: the scripts behind the paper’s Sections 13–16.
- Changing the code: commit with `git` in `~/code/MuonCollider` and `git push` to GitHub; each run records the commit it used.